

lamet-agent 仓库分析报告：

LaMET 高统计测量自动化 agent 的主体结构、各部分关系与项目思路

opencode analy

2026 年 8 月 17 日

摘要

本报告对格点 QCD 自动化分析库 lamet-agent（位于 /root/PyQCD/refer/git-rep/lamet-agent/）进行完整只读分析。该库是“Lattice Measurement with high statistics”思想的 Python 实现：用 LLM agent 自动驱动 LaMET（大动量有效理论）数据分析全流程（关联函数分析、重整化、傅里叶变换、微扰匹配、外推）。全库 98 个文件（78 Python + 13 Markdown + 2 JSONC + 5 其它），Python 46,153 行。报告从三视角展开：**主体结构**（入口 lamet_agent/__main__.py 与 agent.py、核心 core/+planning/+ 五阶段 stages/、知识库 papers/lamet_kb/、示例 examples/、测试 tests/）；**各部分关系**（planning 规划 → stages 执行 → core 测量基元 → 知识库支撑的依赖链）；**项目思路**（把“关联函数分析与外推需要物理专家判断”的步骤委托给 agent，其余固定步骤软件自动化，见 PLAN.md）。主题分析按 **A 代码工作 15 步框架**（A1–A15）重现“agent 如何自动完成一次高统计测量”的完整流程。实测证据：CLI 帮助输出、manifest 校验、23/117/121 项单元测试通过，另有 48 项测试因 ‘examples/*.json’ 可运行 manifest 未随本 checkout 提供而失败（如实记录）。产物 PDF 通过 Overfull 与 Float too large 双零校验。

目录

1 仓库概览	2
1.1 目录结构	3
1.2 规模统计与 git 信息	3
1.3 参考资料引入声明	4
2 主体结构	4
2.1 分层架构	4
2.2 主体结构小结	5
3 各部分关系	5
3.1 依赖主链	5
3.2 对象映射（代码符号 物理/业务对象）	6
3.3 版本演进关系	6
3.4 各部分关系小结	7

4 项目思路	7
4.1 设计意图	7
4.2 演进脉络	7
4.2.1 阶段划分 (数据来源: PROJECT_LOG.md 全文)	7
4.2.2 演进的内在逻辑	8
4.3 典型 workflow	8
5 主题分析: LAMET agent 测量全流程 (A 代码工作 15 步框架)	9
5.1 A1 目的与前期准备	9
5.2 A2 输入是什么	10
5.3 A3 输出应该是什么	10
5.4 A4 整体框架	10
5.5 A5 关键细节	11
5.6 A6 任务如何正确执行	12
5.7 A7 实际执行情况	12
5.8 A8 实际输出情况	13
5.9 A9 结果分析	13
5.10 A10 综合分析	14
5.11 A11 结尾总结	14
5.12 A12 结尾评价	14
5.13 A13 补充说明	14
5.14 A14 参考源	15
5.15 A15 附录与附件 (如有)	15
6 关键代码/文档片段	15
6.1 agent 工具循环 (agent.py)	15
6.2 manifest job-DAG 校验 (manifest.py)	16
6.3 EnsembleData 数据容器 (core/data.py)	16
6.4 LLM 提供商注册表 (core/llm.py)	17
6.5 3pt/2pt 比值谱分解模型 (stages/correlator/functions.py)	17
6.6 自重整化拟合函数 (stages/renorm/functions.py)	17
6.7 匹配矩阵构造 (kernels.py)	18
6.8 规划工具目录 (planning/agent.py)	19
7 参考源清单	19
8 结论与局限	22

1 仓库概览

1.1 目录结构

lamet-agent 采用“根级包 + 子包分层”布局：入口文件在仓库根，核心逻辑分 core/（通用基元）、planning/（交互规划）、stages/（五阶段 + 评审）三组，知识库与示例并列在 papers/、examples/ 下。实测目录树如下（数据来源：find 实测）：

```

1 lamet-agent/
2 |-- README.md / PLAN.md / PROJECT_LOG.md / pyproject.toml / .gitignore
3 |-- docs/                # 报告产物 (analy_lamet_20260817 为本报告)
4 |-- examples/
5 |   |-- sample_manifest.jsonc / partial_sample_manifest.jsonc
6 |   |-- fake_data/        # generate_fake_data.py / read_fake_data.py / utils.py
7 |-- lamet_agent/
8 |   |-- __main__.py / agent.py / manifest.py / manifest_params.py / kernels.py
9 |   |-- core/             # data llm tools resampling prompting plotting reporting
10 |   |   |                 # trace banner stages
11 |   |-- planning/        # core.py agent.py questions.py render.py conversion.py
12 |   |-- stages/         # correlator renorm fourier matching extrapolation review
13 |-- lamet_literature/    # arxiv.py classify_arxiv.py inspirehep.py README
14 |-- papers/
15 |   |-- README.md / docs/ # lamet_rubric.md schema.md
16 |   |-- lamet_kb/        # pipeline.py scoring.py storage.py arxiv.py settings.py
17 |-- tests/unit/         # 21 个测试文件 (460 个测试函数)

```

1.2 规模统计与 git 信息

指标	实测值
总文件数	98 (含.tex/.pdf 先前 pure 产物 2 个)
Python 文件数	78 (find 实测, 46,153 行)
Markdown 文档数	13 (3,050 行)
JSONC 清单数	2 (sample_manifest.jsonc、partial_sample_manifest.jsonc)
单元测试函数	460 (grep -c "def test_" 实测)
stage 包数	6 (correlator/renorm/fourier/matching/extrapolation/review)
底层目录	20
提交数	该库 非独立 git 仓库 ：被 PyQCD 主仓库跟踪，本目录相关提交 2 条
标签数	主仓库标签（沿用主仓库历史，不体现 lamet-agent 自身里程碑）

表 1: 仓库实测统计（数据来源：find/wc/git 实测）

git 说明 (确证): git -C lamet-agent log 返回的是主仓库历史 (HEAD 为 e953b47)，本目录仅有两笔直接提交 2daadb3 (2026-08-16) 与 4d84e9c (2026-08-16)。因此本报告以仓库内的 PLAN.md、PROJECT_LOG.md、README.md 作为演进脉络主证据，git 历史只作辅助。

1.3 参考资料引入声明

按 `analy` 技能默认引入参考知识库：`examples/`（5 文件）作为示例层引入；`papers/docs/`（2 文件：`lamet_rubric.md`、`schema.md`）作为知识库规范引入；`lamet_literature/`（`README` + 3 脚本）作为文献标注子库引入。`docs/` 下已有先前 `pure` 产物 `pure_core_20260815.tex/pdf`，本报告只读不覆盖。

2 主体结构

2.1 分层架构

`lamet-agent` 的主体按“入口层 → 核心层 → 知识库/示例/测试层”组织，核心层内部再分规划（`planning`）、执行管线（`stages`）、通用基元（`core`）三组。分层表如下：

层次	目录	职责一句话
入口层	<code>lamet_agent/__main__.py</code>	CLI 入口： <code>validate/plan/run</code> 三命令，负责 <code>manifest</code> 解析、后端解析与 <code>plan</code> 回退（ <code>__main__.py</code> :111-276）
入口层	<code>lamet_agent/agent.py</code>	<code>agent</code> 运行时：执行 <code>metadata.stages</code> 有序 <code>stage</code> ，为每个 <code>job</code> 建独立 <code>store</code> 并驱动 LLM/工具循环（ <code>agent.py</code> :561-1012）
核心层	<code>lamet_agent/planning/</code>	交互式 <code>manifest</code> 规划： <code>JSONC</code> 宽松解析、确定性问题检查、LLM 驱动的 <code>JSON Patch</code> 候选编辑、 <code>quick/full</code> 清单生成（ <code>planning/core.py</code> :160-175）
核心层	<code>lamet_agent/stages/*/</code>	五阶段执行管线 + <code>review</code> ：每 <code>stage</code> 含 <code>functions.py</code> （工具 + 注册表）、 <code>validation.py</code> 、 <code>prompts.md</code> 、 <code>reporting.py</code>
核心层	<code>lamet_agent/core/</code>	通用基元：数据容器（ <code>data.py</code> ）、LLM 后端（ <code>llm.py</code> ）、工具注册与参数准备（ <code>tools.py</code> ）、重采样（ <code>resampling.py</code> ）、提示组装（ <code>prompting.py</code> ）、作图（ <code>plotting.py</code> ）
核心层	<code>lamet_agent/manifest.py</code>	<code>job-DAG manifest</code> 模式： <code>Pydantic</code> 模型 + 全库 <code>id</code> 唯一性/ <code>DAG</code> /参数契约校验（ <code>manifest.py</code> :385-529）
核心层	<code>lamet_agent/kernels.py</code>	<code>NLO</code> 匹配核库： <code>KERNEL_REGISTRY</code> 注册核函数、 <code>alphas_nloop</code> 、 <code>build_matching_matrix</code> （ <code>kernels.py</code> :58-420）
知识库层	<code>papers/lamet_kb/</code>	<code>LaMET</code> 文献知识库： <code>arXiv</code> 抓管线、相关性打分、 <code>SQLite</code> 存储（ <code>lamet_kb/pipeline.py</code> :26-60）
知识库层	<code>lamet_literature/</code>	文献标注：本地模型给 <code>arXiv HTML</code> 打结构化标签（ <code>classify_arxiv.py</code> ）
示例层	<code>examples/</code>	注释清单 <code>sample_manifest.jsonc</code> 、 <code>fake_data/</code> 假数据生成/读取
测试层	<code>tests/unit/</code>	21 个单元测试文件、460 个测试函数（ <code>pytest</code> 实测）

层次	目录	职责一句话
配置层	pyproject.toml	包元数据、可选依赖组 (analysis/codex/literature)、console script

表 2: 主体结构分层 (数据来源: 全库索引实测)

2.2 主体结构小结

分层要点: 入口层极薄 (只做解析与编排), 全部数值工作下沉到 `stages/*/functions.py` 的工具函数, 由 `core/tools.py` 的注册表统一暴露给 agent; `core/` 是纯 Python 的 LQCD 统计与分析基元, 不依赖 LLM 逻辑即可单独复用; `planning/` 与 `stages/` 共享 `manifest.py/manifest_params.py` 的同一套模式与契约。下一节回答“各部分之间如何连接”。

3 各部分关系

3.1 依赖主链

要点

依赖主链: `__main__.py` -> `agent.py` -> `core/{llm,tools,prompting}.py` -> `stages/*/functions.py` -> `core/{data,resampling}.py` **平行链:** `planning/` -> `manifest.py/manifest_params.py/core/tools.py` **知识库支撑:** `stages/review` -> `papers/lamet_kb` (SQLite)

- **入口 → 核心:** `__main__.py:332-344` 调用 `run_agent`; `run_agent` 在 `agent.py:611-614` 遍历 `manifest.metadata.stages`, 每个 stage 再遍历其 jobs。
- **stage 路由:** `core/stages.py` 维护 `STAGE_TO_PACKAGE` 映射 (`core/stages.py:6-16`), 把 stage id 映射到 stage 包名; `core/tools.py` 的 `resolve_stage_tools/resolve_job_tools` 加载该包 `STAGE_TOOLS` 并做 job 级过滤 (`core/tools.py:199-288`)。
- **数据流:** correlator stage 产出裸矩阵元 `EnsembleData` `NetCDF`, 再由 `renorm` 消费 (`target/denominator/reference` 角色)、`fourier` 消费 (`input` 角色)、`matching` 消费 (`quasi` 角色)、`extrapolation` 消费 (`lightcone` 角色); `agent.py` 用 `outputs` 字典把 job id 关联到 `store["output"]` (`agent.py:657-659`)。
- **知识库支撑:** `stages/review/functions.py` 的 `write_review_from_manifest` 读 SQLite 检索文献并按 manifest 物理锚点加权排名 (`review/functions.py:293-399`); `papers/lamet_kb` 的 `harvest_lamet` 管线负责填充该库 (`lamet_kb/pipeline.py:26-60`)。

- **planning 复用**: `planning/core.py` 的 `check_manifest_draft` 调 `AnalysisManifest.model_validate` (`planning/core.py:842-843`) , `validate_candidate_payload` 复用严格校验 (`planning/core.py:1534-1539`), 实现 “同一模式、两处消费”。

3.2 对象映射 (代码符号 物理/业务对象)

代码符号	对象概念	物理/业务含义
<code>CorrelatorInput</code>	一条 2pt/3pt 关联函数数据集	LQCD 两点/三点关联函数及其运动学 (momentum/volume/tsep/bz/bT)
<code>EnsembleData</code>	重采样后的集合数据容器	每样本一个 resample 轴 (bootstrap/jackknife/raw/g-var) + 物理坐标 (z 或 x)
<code>pt3_ratio_fcn</code>	3pt/2pt 比值模型	$C_{3pt}(t, \tau)/C_{2pt}(t)$ 谱分解比值, 含 O_{ij} 矩阵元与能级 E_n
<code>fit_self_renormalization_factor</code>	自重整化因子 z_R 拟合	$\ln M(z, a)$ 线性发散 + 对数发散 + 离散效应的拟合 (PLAN.md 公式)
<code>run_fourier_transform</code>	坐标空间 $\rightarrow$ 动量空间傅里叶变换	asymptotic extrapolation + 实部对称/虚部反对称 convention 的准分布重建
<code>build_matching_matrix</code>	微扰匹配矩阵	LaMET 核 $\tilde{q}(x; P_z) \rightarrow q(x)$ 的列求和 plus 处方离散化
<code>run_extrapolation</code>	连续极限/无穷体积/手征极限外推	对 a^2 、 $1/P_z$ 、 m_π 等外推项的多项式拟合
<code>STAGE_TOOLS</code>	stage 工具注册表	agent 可调用的数值工具集合 (inspect/tune/fit/apply/-plot/report)
<code>random_seed</code>	全局重采样种子	每个 bootstrap/jackknife 步骤的 RNG 种子 (manifest 权威配置)

表 3: 对象映射表 (数据来源: 源码与 README 实测)

3.3 版本演进关系

- **manifest 契约三次迁移**: 初始 “顶层 CLI+manifest+kernels” 扁平布局 (PROJECT_LOG.md:5-17) $\rightarrow$ “core+ 五 stage 包” 运行布局 (PROJECT_LOG.md:21-28) $\rightarrow$ “metadata+ 全局 inputs+ 每

stage defaults/jobs” 的 job-DAG 契约 (PROJECT_LOG.md:343-349)。

- **correlator 工具 12→4 收敛**: 2026-06-10 重写 stages/correlator/functions.py, 把 12 个底层工具收敛为 inspect_correlator_scale/tune_ground_state/tune_bare_matrix/fit_bare_matrix_grid 四个 agentic 工具 (PROJECT_LOG.md:273-281)。
- **提示词资源化**: 2026-08-11 把各 stage 提示词移入 prompts.md Markdown 资源, skills.py 更名为 validation.py (PROJECT_LOG.md:976-1005)。
- **包布局迁移**: src/lamet_agent/ 迁至根级 lamet_agent/ (PROJECT_LOG.md:1009-1016)。

3.4 各部分关系小结

关系要点: 三组核心模块共享 manifest.py 的 job-DAG 模式——stages/ 消费它执行, planning/ 消费它规划, review/ 消费它做一致性检查; core/ 是唯一被三组共同依赖的基元层; 知识库 (papers/lamet_kb/) 只向 review 提供“背景文献”, 不参与数值计算 (review/functions.py:503-516 中“literature 仅 background only”约束)。

4 项目思路

4.1 设计意图

PLAN.md 明确点出设计动机: “关联函数分析和外推步骤需要物理学家的专业知识来设计分析策略, 而其他步骤相对固定, 可以通过软件工具自动化完成。因此, 将这两个步骤委托给智能体可以显著提高分析效率” (PLAN.md:12)。由此形成三层分工:

- **智能决策层** (correlator analysis、extrapolation): agent 依据数据特点选择拟合函数、拟合区间、贝叶斯先验、外推项组合 (PLAN.md:14-16、PLAN.md:142-144);
- **固定流程层** (renorm、fourier、matching): 相对固定的计算流程, agent 只负责“检查输入是否完整并稳定执行”, 不自行决定物理方案 (PLAN.md:44、PLAN.md:140);
- **统计基础层**: 全程保留重采样 (bootstrap/jackknife) 用于统计误差, 并考虑不同方案的系统误差 (PLAN.md:16、PLAN.md:144)。

why 问到底: 为什么用 LLM agent 而不是传统脚本? 因为关联函数分析的“策略选择空间”过大 (multi-state joint fit / summation method / model averaging / GEVP 等, 各有权衡, 见 PLAN.md:24-29), 传统硬编码脚本无法覆盖; 而 LLM 恰好擅长“根据证据选择策略”。因此 agent 的数值动作全部限制在 STAGE_TOOLS 注册表内 (core/tools.py:199-204), LLM 只做“决策层”, 不直接执行数值运算——这是“可控 agent”与“自由 agent”的边界设计。

4.2 演进脉络

4.2.1 阶段划分 (数据来源: PROJECT_LOG.md 全文)

阶段	时间区间	关键演化
脚手架	2026-05-31	最小非 temp 项目脚手架, 扁平布局 (cli/manifest/kernels)
五 stage 骨架	2026-06-01	拆出 core/ 与五 stage 包
correlator 首个实作	2026-06-03	复制 LaMETLat 数值(read_pt2/bootstrap/jackknife/pt2 fit) + 新增 scan_tmin/model_average
agent 循环成型	2026-06-03	mock/external/deepseek 后端、工具执行循环、 core/tools.py、JSON 总结
agentic 收敛	2026-06-10	correlator 12 工具 → 4 工具, inspect → tune → fit 流程
NetCDF 中间产物	2026-06-18	EnsembleData.to_netcdf/from_netcdf(xarray NETCDF4)
job-DAG manifest	2026-06-20	metadata/inputs/stages 三块契约 + 每 job 独立 store
全局重采样元数据	2026-07-01	random_seed/bs_samples/bin_size/sample_error_mode/ workers 上移 metadata
交互规划	2026-07-07	lamet-agent plan: LLM 驱动 JSON Patch 候选编辑 + 确定性 guardrail
自重整化	2026-07-08-18	self_renormalization 方案 (fit z_R + apply $H/(z_R Z_{MS})$)
HDF5 v2 契约	2026-07-15	标准 correlator HDF5 布局、操作符标签自由化、离散运动学权威化
提示词资源化	2026-08-11	prompts.md Markdown + skills.py → validation.py
文献标注	2026-08-13	classify_arxiv.py 本地模型给 128 篇 arXiv 打结构化标签

表 4: 演进脉络表 (数据来源: PROJECT_LOG.md 实测)

4.2.2 演进的内在逻辑

一条主线: 从“CLI 骨架”到“LLM 驱动的可规划分析 agent”，每一步都围绕 **让 agent 更可控、让物理更可追溯** 两个目标。典型证据: (1) 全局重采样元数据上移 metadata, 是为了让“统计配置单一来源、agent 不可越权修改” (README.md:337-340); (2) 严格 stage 参数契约 (manifest_params.py) 拒绝未知键, 是为了防 agent 拼错参数 (PROJECT_LOG.md:644-648); (3) manifest 校验失败自动进入 plan 模式, 是为了“校验失败也能借 LLM 修复, 而非硬中断” (PROJECT_LOG.md:1049-1060)。

4.3 典型 workflow

agent 如何自动完成一次高统计测量 (结合 README.md:931-959 与实测):

- 准备:** 用户编写 JSONC manifest (examples/sample_manifest.jsonc), 声明 metadata、输入关联函数、核函数、每 stage 的 defaults/jobs;
- 规划 (可选):** lamet-agent plan draft.jsonc (完整参数见 README.md:592-595) 让 LLM 补全/修复草稿, 经 JSON Patch 候选校验后生成 quick/full manifest;
- 校验:** lamet-agent validate 执行 schema + DAG + 参数契约 + 路径检查 (__main__.py:111-135);

4. **运行**: `lamet-agent run manifest.json --backend api` 进入 `run_agent`, 按 `metadata.stages` 顺序执行; 每 stage 每 job 建独立 store, 静态提示一次性组装 (`core/prompting.py:64-100`), 此后每轮把工具观察追加为新的 user turn;
5. **工具循环**: LLM 每轮输出一个 JSON action (`call_tool/request_user_input/finish`), `agent.py` 校验必需工具序列 (`required_job_tool_sequence`), `core/tools.py` 的 `prepare_tool_args` 注入 manifest 路径/绘图路径/随机种子等参数 (`core/tools.py:357-372`);
6. **阶段产物**: 每 stage 的终端工具把主数据放 `store["output"]`, 随后写 NetCDF 工件与双语文档报告 (`correlator`→`renorm`→`fourier`→`matching`→`extrapolation` 逐级手递, `agent.py:857-989`);
7. **评审**: review 阶段读五个 stage 报告 + 文献背景, 产出物理总结 + manifest 修改建议 (`review/functions.py:578-585`)。

项目思路

项目思路一句话: lamet-agent 把“需要物理专业判断的分析步骤”委托给 LLM agent, 用 manifest job-DAG 做执行蓝图、用 `core/` 基元做数值底座、用 `planning/` 做安全护栏, 最终把人工 LQCD 数据分析流程压缩为“一次 manifest 驱动的高统计测量自动化”。

5 主题分析: LAMET agent 测量全流程 (A 代码工作 15 步框架)

本节按 `analy` 技能 A (代码/软件工作) 15 步框架完整重现“agent 自动完成一次高统计测量”这一具体工作。判定为代码工作: 对象是以 manifest 驱动的 agent 软件流程。

5.1 A1 目的与前期准备

目的: 把 LaMET 数据分析 (`correlator analysis` → `renormalization` → `fourier` → `matching` → `extrapolation`) 自动化, 让 agent 代替物理学家完成策略选择与数值执行 (`PLAN.md:5-12`)。

前置条件:

- Python ≥ 3.10, 安装可选依赖组: `pip install -e ".[dev,analysis]"` (`analysis` 含 `gvar/lscfit/scipy/h5py/matplotlib/xarray/netCDF4/mpmath`, `pyproject.toml:20-28`);
- 标准 correlator HDF5 数据 (2pt/3pt 布局见 `README.md:669-711`), 或可经 `plan` 模式转换;
- 可选: LLM API key (`api.key` 或 `DEEPSEEK_API_KEY/OPENAI_API_KEY`, `core/llm.py:44-55`)。

实测确认: 本机 `python3 -c "import lamet_agent"` 成功 (版本 0.1.0), `analysis` 依赖全部可导入。

5.2 A2 输入是什么

主要输入：manifest 文件 (JSON/JSONC) 。三块结构：metadata (run 级设置：run_id/root_directory/artifacts_directory/target_observable/parton/resample_mode/random_seed/stages 等, manifest.py:71-96) 、inputs (correlators/artifacts/kernels 三类源节点池, manifest.py:355-360)、stages (每 stage 的 defaults + jobs 列表, manifest.py:375-383)。

派生输入：HDF5 关联函数文件 (由 correlators[].data_path 指向)、外部 NetCDF 工件 (artifacts)、核函数文件 (kernels[].kernel_path, 默认 lamet_agent/kernels.py)。

示例 (examples/sample_manifest.jsonc:28-45 片段直引):

```

1  "metadata": {
2    "run_id": "sample_run_id",
3    "root_directory": "..", // resolve paths from the repository root
4    "artifacts_directory": "examples/artifacts", // relative to the root directory for storing artifacts
5    "target_observable": "pdf", // "pdf", "da" or "gpd"
6    "parton": "quark", // "quark" or "gluon"
7    "resample_mode": "jk", // "jk" or "bs" — shared by correlator and fourier stages
8    "sample_error_mode": "covariance", // "mean", "median", or "covariance"; "median" requires resample_mode "bs"
9    "random_seed": 1984, // required; RNG seed for every resampling step in the run
10   "workers": 1, // optional positive integer; sample-fit worker processes (default: 1, serial)
11   // "bs_samples": 500, // REQUIRED when resample_mode is "bs"; no default — must be set explicitly
12   // "bin_size": 1, // optional; bins configurations before jackknife/bootstrap resampling (default: no binning)
13   // Global stage ORDER only. Which data sets flow through each stage is
14   // decided by the jobs declared under "stages", not here. Optional terminal
15   // review can be appended as "review" when a review job is also declared with
16   // "defaults": {"literature": true, "literature_max_papers": 4}.
17   "stages": ["correlator_analysis", "renormalization", "fourier_transform", "perturbative_matching", "extrapolation"
18   ↪ ]
19 },

```

5.3 A3 输出应该是什么

- **中间产物：**每 stage 的 EnsembleData NetCDF 工件, 路径 <artifacts_directory>/<stage>/<job_id>.nc (README.md:73-96);
- **图表：**2pt fit-on-data、ratio 拟合带、重整化矩阵元、quasi 分布、外推/系统误差图 (PDF/SVG);
- **报告：**每 stage 双语文档报告 (ca_report.md/renorm_report.md/ft_report.md/..., 见 agent.py:857-989), review 阶段产出完整评审 Markdown;
- **最终结果：**光锥分布函数数组 (.npy 或 NetCDF) + 物理图 + 统计/系统误差。

5.4 A4 整体框架

模块组织与主流程 (结合 agent.py:561-1012 实测):

要点

主 流 程: CLI run → validate → run_agent → for stage in metadata.stages → for job in jobs → hydrate artifacts → build static prompt → LLM/tool loop → store['output'] → stage report

```

1  selected = list(manifest.metadata.stages)
2  state = AgentState(run_id=manifest.run_id)
3  session = make_llm_session(
4      backend,
5      actions_path,
6      api_key=api_key,
7      provider=provider,
8      model_name=model_name,
9      base_url=base_url,
10 )
11 trace = AgentTrace(enabled=verbose, quiet_ui=not verbose)
12 outputs: dict[str, Any] = {item.id: item for item in manifest.inputs.artifacts}
13 stage_reports: dict[str, dict[str, str]] = {}
14 if backend == "api" and provider and model_name:
15     model_spec = format_api_model_spec(provider, model_name)
16 elif backend == "codex":
17     model_spec = model_name
18 else:
19     model_spec = None
20
21 if verbose:
22     trace.run_begin(
23         run_id=manifest.run_id,
24         backend=backend,
25         stages=selected,
26         model_spec=model_spec,
27     )
28 else:
29     trace.run_banner(
30         run_id=manifest.run_id,
31         backend=backend,
32         stages=selected,
33         model_spec=model_spec,
34     )
35 for stage in selected:
36     state.stage_results[stage] = {}

```

5.5 A5 关键细节

关键函数与易错点:

- **job store 隔离**: 每 job 新建 store 字典, 上游 job 输出按角色注入 (agent.py:623-635), 下游引用缺失即抛错——防止“洞”进入下游 stage (PROJECT_LOG.md:794-802);
- **必需工具序列**: required_job_tool_sequence 按 stage+job 角色给出必须依次成功的工具, agent 在序列完成前不得 finish (agent.py:194-207、core/tools.py:250-288);

- **工具参数准备**: prepare_tool_args 从 manifest 注入路径/选择器/重采样种子, 并把绘图路径改写成 stage 工件目录 (core/tools.py:357-372);
- **重采样正确性**: core/resampling.py 提供共享 bootstrap indices 与 leave-one-out jackknife, 要求 2pt/3pt 用同一组 indices 再构成比值 (PROJECT_LOG.md:268-271);
- **复数 NetCDF**: auto_complex=True 保证复数矩阵元直接 round-trip (core/data.py:148-168)。

```

1 def bootstrap(data: np.ndarray, n_samples: int, axis: int = 0, seed: int | None = 1984, bin_size: int = 1) -> np.
    ↳ ndarray:
2     """Generate bootstrap sample averages from ensemble data."""
3     data = np.asarray(data)
4     if bin_size > 1:
5         data = bin_data(data, bin_size, axis=axis)
6     n_conf = data.shape[axis]
7     rng = np.random.default_rng(seed)
8     indices = rng.choice(n_conf, (n_samples, n_conf), replace=True)
9     return np.take(data, indices, axis=axis).mean(axis=axis + 1)
10
11
12 def jackknife(data: np.ndarray, axis: int = 0, bin_size: int = 1) -> np.ndarray:
13     """Generate leave-one-bin-out jackknife sample averages from ensemble data."""
14     data = np.asarray(data)
15     if bin_size > 1:
16         data = bin_data(data, bin_size, axis=axis)
17     n_conf = data.shape[axis]
18     total = data.sum(axis=axis, keepdims=True)
19     return (total - data) / (n_conf - 1)

```

5.6 A6 任务如何正确执行

正确命令序列 (数据来源: README.md:575-595 实测):

```

1 python3 -m venv .venv && source .venv/bin/activate
2 pip install -e ".[dev,analysis]"
3 lamet-agent validate examples/sample_manifest.jsonc
4 lamet-agent run examples/sample_manifest.jsonc --backend api --model deepseek/deepseek-chat --verbose
5 # 或 plan 草稿:
6 lamet-agent plan draft_manifest.jsonc --backend api --model deepseek/deepseek-chat
7 # mock 冒烟 (无 LLM) :
8 lamet-agent run examples/sample_manifest.jsonc --backend mock

```

执行要点: manifest 的 root_directory 必须解析到 lamet-agent checkout 根 (manifest.py: 666-682); 数据/工件/核路径必须存在 (除 artifacts 输出目录可后建)。

5.7 A7 实际执行情况

实测命令与输出 (本次分析会话实测, 记录于会话日志):

1. python3 -m lamet_agent --help: 成功, 输出 validate/plan/run 三个子命令及 CLI-first scaffold 说明 (确证);

2. `python3-mlamet_agentvalidateexamples/sample_manifest.jsonc`: 失败——报 `inputs.correlators[0].data_pathdoesnotexist`, 因样例清单指向的 `data_pion_pdf_cg/...` 真实数据未随本 checkout 提供 (确证, 非代码缺陷);

3. `pytest` 子集实测:

- `test_stage_core/test_banner/test_resampling/test_io`: **23 passed** (1.22s, 确证);
- `test_schemas/test_validation/test_stage_core`: **117 passed** (0.71s, 确证);
- `test_agent + test_planning`: **121 passed, 2 failed** (3.40s, 确证);
- 其余 unit (排除 `planning/correlator/fourier` 三大件): **276 passed, 48 failed** (12.79s, 确证)。

失败原因归因 (确证): 48 项失败大多在 `test_tools.py/test_agent.py`, 根因是测试直接加载 `examples/*.json` (如 `pion_pdf_cg_manifest.json`、`pion_da_gi_manifest.json`), 而本 checkout 仅跟踪 `*.jsonc` 模板——可运行 `manifest` 未入库。这是“仓库快照不完整”, 并非被测代码必然缺陷; `PROJECT_LOG.md` 亦提示 2026-08-11 审计时“529-test audit leaves two failures”的历史状态。

5.8 A8 实际输出情况

实测产物: 本次分析不执行真实数值 `run` (无真实数据), 仅验证 (1) 包可导入 (`lamet_agent 0.1.0`); (2) CLI 可用; (3) 测试子集绿/红统计。**工程内声明的预期产物** (`README.md:73-96`): `correlator` 裸矩阵元 `ca_p5.nc`、重整化 `rn_p5.nc`、傅里叶 `fourier_result.nc`、匹配 `quasi_pdf.nc`。这些为仓库声明输出, 本次无数据未验证 (**未验证**)。

5.9 A9 结果分析

结合实测解读:

- 23/117 通过的两组测试覆盖数据容器、重采样、提示组装、schema 校验——说明**数值基元与模式层稳定** (确证);
- `test_agent` 的 2 项失败均为“加载缺失 `manifest`”路径错误 (`ValueError: Manifest does not exist`), 非逻辑断言失败——说明 `agent` 循环逻辑本身在缺失数据下仍能按预期进入错误分支 (确证);
- 48 项失败集中于 `test_tools.py` 的参数准备测试——这批测试依赖具体 `example manifest` 的参数值 (`PROJECT_LOG.md:637` 已提示“decoupled tool-preparation tests from mutable example-manifest parameter values”是维护方向)。

结论分级: 数值基元稳定 (确证); `agent` 循环在缺失 `manifest` 时报错路径正确 (确证); 完整真实 `run` 未执行, 其物理正确性**未验证**。

5.10 A10 综合分析

与仓库整体联系：A1–A9 覆盖了 `planning→manifest→agent→core→stages→artifacts` 的完整引用链，印证第二节“依赖主链”。**代码-对象映射：**`fit_bare_matrix_grid` 从 3pt/2pt 比值提取裸矩阵元 O_{00} ；`apply_self_renormalization` $H_R(z) = H_{bare}(z)/(z_R Z_{MS}(z))$ ；`run_fourier_transform` 坐标空间矩阵元 $\tilde{h}(z, P_z)$ 到动量空间准分布。**深层原因：**为什么 agent 流程能成立？因为 LaMET 流程的五步具有“上游数值确定性”——每步输出是标准 NetCDF (EnsembleData)，下游只消费该契约；LLM 的作用被限制在“选择策略”而非“改写数值”，所以 manifest 校验 (`validate_dag`) 能兜底整个 DAG 的正确性。

5.11 A11 结尾总结

结论

“LAMET agent 测量全流程”是一条 `manifest` (蓝图) → `planning` (可交互规划) → `run_agent` (编排) → 五 `stage` 工具循环 (执行) → NetCDF/报告 (产物) → `review` (评审) 的闭环。其设计核心是“LLM 决策 + 确定性数值 + 模式校验”三者分工：LLM 只在注册表内选动作，数值全部走 `core/` 与 `stages/` 的确定性函数，manifest 的 `schema/DAG/` 参数契约对每次变更都做严格校验。

5.12 A12 结尾评价

优点：(1) 数值与决策解耦，可离线测试 mock 后端与外部回放 (`core/llm.py:417-431`)；(2) `job-DAG` + 每 `job` 独立 `store` 让部分运行 (resume) 天然成立 (`README.md:651-652`)；(3) 中间产物 NetCDF 自描述，用户可逐 `stage` 检查。**缺点/可改进：**(1) 本 checkout 缺少可运行 `example manifest`，导致大量集成测试无法跑通——仓库应补齐 `examples/*.json` 或把测试改为生成式清单；(2) LLM 每轮只出一个 `action` 的循环在长任务下 `token` 消耗高，静态提示 + 增量观察的设计只能缓解不能根除 (`PROJECT_LOG.md:86-89`)；(3) `review` 依赖 LLM 文本输出，数值断言依赖 `deterministic checks` (`review/functions.py`)，二者一致性靠提示约束，缺乏形式化校验。

5.13 A13 补充说明

假设/局限/未覆盖：

- 未执行真实数值 `run` (无真实 LQCD 数据随仓库提供)，A8 中物理产物清单为仓库声明 (未验证)；
- `examples/*.json` 可运行 `manifest` 未随本 checkout 提供 (确证)，真实跑通需向作者索取 `data_pion_pdf_cg/` 等数据或自行用 `fake_data` 生成；
- 分布式/GPU 相关内容 (`pyqcd.parallel` 等) 不属于本库，本库为单机串行 + 可选多进程 `workers`。

5.14 A14 参考源

本节全部证据已并入第 7 节参考源清单；核心引用：PLAN.md:5-16、README.md:931-959、agent.py:561-1012、core/tools.py:199-372、core/resampling.py:63-195、manifest.py:385-529、examples/sample_manifest.jsonc:28-45、pyproject.toml:10-28。

5.15 A15 附录与附件（如有）

无独立附件文件；实测命令与完整输出记录于会话日志 `.analy.<ts>.log`。仓库内可参考的补充资料：README.md（NetCDF 格式、manifest 语义、自重整化章节）、PROJECT_LOG.md（逐日演进）。

6 关键代码/文档片段

以下片段全部用 `\lstinputlisting` 直引仓库原文件（每段 40 行，行号实测）。

6.1 agent 工具循环 (agent.py)

```

1  for cycle in range(1, max_tool_steps + 1):
2      trace.cycle_begin(cycle)
3      if last_observation is not None:
4          trace.prompt_delta(last_observation)
5      trace.llm_call_begin(backend=backend, model_spec=model_spec)
6      action = session.decide(last_observation=last_observation)
7      trace.llm_call_end()
8      trace.model_output(action)
9      state.actions.append({"stage": stage, "job": job.id, "action": action})
10
11     if action.get("action") != "call_tool":
12         if required_index < len(required_sequence):
13             expected = required_sequence[required_index]
14             observation = {
15                 "tool_name": expected,
16                 "error": (
17                     f"job {job.id!r} cannot {action.get('action', 'stop')} before "
18                     f"required tool {expected!r} succeeds"
19                 ),
20             }
21             observations.append(observation)
22             trace.observation(observation)
23             last_observation = observation
24             continue
25     if action.get("action") == "request_user_input":
26         args = action.get("args") or {}
27         questions = action.get("questions")
28         if questions is None:
29             prompt = args.get("prompt") or action.get("reason") or "User input requested."
30             questions = [prompt] if isinstance(prompt, str) else list(prompt)
31         elif isinstance(questions, str):
32             questions = [questions]
33         else:

```



```

34         questions = list(questions)
35         state.pending_user_input.setdefault(stage, {})[job.id] = questions
36         break

```

6.2 manifest job-DAG 校验 (manifest.py)

```

1  @model_validator(mode="after")
2  def validate_dag(self) -> "AnalysisManifest":
3      parameter_issues: list[str] = []
4      for stage, config in self.stages.items():
5          parameter_issues.extend(
6              validate_stage_parameter_mapping(
7                  stage,
8                  config.defaults,
9                  path=f"stages.{stage}.defaults",
10             )
11         )
12         for index, job in enumerate(config.jobs):
13             parameter_issues.extend(
14                 validate_stage_parameter_mapping(
15                     stage,
16                     job.params,
17                     path=f"stages.{stage}.jobs[{index}].params",
18                 )
19             )
20     if parameter_issues:
21         details = "\n".join(f"- {issue}" for issue in parameter_issues)
22         raise ValueError(f"Unsupported stage manifest parameters:\n{details}")
23
24     for index, kernel in enumerate(self.inputs.kernels):
25         if "scheme" in (kernel.model_extra or {}):
26             raise ValueError(
27                 f"inputs.kernels[{index}].scheme is no longer supported; "
28                 f"move it to stages.{kernel.stage}.defaults.scheme or the "
29                 f"corresponding jobs[].params.scheme"

```

6.3 EnsembleData 数据容器 (core/data.py)

```

1  class EnsembleData:
2      def __init__(
3          self,
4          ensemble: EnsembleInfo,
5          resample: ResampleType,
6          values: Union[List[Union[int, float, complex, NDArray]], gvar.GVar, NDArray[gvar.GVar]],
7          dims: DimsType,
8          coords: CoordsType,
9          attrs: Optional[Dict[str, str]] = None,
10         name: Optional[str] = None,
11     ) -> None:
12         if resample not in RESAMPLE_TYPE_VALUES:
13             raise ValueError(f"Unknown resampling method '{resample}'.")
14         self.resample: ResampleType = resample

```

```

15     self.ensemble = ensemble
16     self.array = self._build_xarray(resample, values, dims, coords, attrs, name)
17
18     @staticmethod

```

6.4 LLM 提供商注册表 (core/llm.py)

```

1 PROVIDERS: dict[str, dict[str, str]] = {
2     "deepseek": {
3         "base_url": "https://api.deepseek.com",
4         "default_model": "deepseek-v4-flash",
5         "key_env": "DEEPSEEK_API_KEY",
6     },
7     "openai": {
8         "base_url": "https://api.openai.com/v1",
9         "default_model": "gpt-4o-mini",
10        "key_env": "OPENAI_API_KEY",
11    },
12 }

```

6.5 3pt/2pt 比值谱分解模型 (stages/correlator/functions.py)

```

1 def pt3_ratio_fcn(
2     t: np.ndarray,
3     tau: np.ndarray,
4     p: dict,
5     Lt: int,
6     *,
7     nstate: int = 2,
8     part: str = "re",
9 ) -> np.ndarray:
10     """Real (``part='re'``) or imaginary (``part='im'``) n-state 3pt/2pt ratio."""
11     energies = _state_energies(p, nstate)
12
13     numerator = 0.0
14     for src, src_e in enumerate(energies):
15         for snk, snk_e in enumerate(energies):
16             matrix_element = p[f"0{min(src, snk)}{max(src, snk)}_{part}"]
17             numerator = numerator + (
18                 matrix_element
19                 * p[f"z{src}"]
20                 * p[f"z{snk}"]
21                 * np.exp(-src_e * (t - tau))
22                 * np.exp(-snk_e * tau)
23                 / (2 * src_e)
24                 / (2 * snk_e)
25             )
26     return numerator / pt2_re_fcn(t, p, Lt, nstate=nstate)

```

6.6 自重整化拟合函数 (stages/renorm/functions.py)

```

1  def fcn(z_x_in, p):
2      out_vals = []
3      for zm, xm in zip(z_x_in["z"], z_x_in["x"]):
4          out_vals.append(
5              SELF_RENORM_K * zm * xm / gv.log(lambdaqcd_gev / xm)
6              + p[f"g{zm}"]
7              + p[f"f1{zm}"] / xm
8              + 3 * SELF_RENORM_CF / SELF_RENORM_BO
9              * gv.log(
10                 gv.log(xm / lambdaqcd_gev)
11                 / gv.log(mu / lambdaqcd_gev)
12             )
13              + gv.log(1 + d_val / gv.log(lambdaqcd_gev / xm))
14          )
15      return out_vals

```

6.7 匹配矩阵构造 (kernels.py)

```

1  def build_matching_matrix(
2      lc_x_ls: np.ndarray,
3      mu: float,
4      quasi_y_ls: np.ndarray | None,
5      eps: float,
6      *,
7      density: DensityFn,
8      color_factor: float = CF,
9      diagonal_extra: Callable[[float], float] | None = None,
10 ) -> np.ndarray:
11     """Discretize a kernel density ``density(x, y)`` into an (nx, ny) matching matrix.
12
13     This is the column-sum plus prescription: it knows no physics beyond "a regular NLO
14     integrand singular on  $x = y$ , made a plus function by forcing each  $y$  column to sum to
15     zero". It serves the meson-DA kernel, a genuine two-variable  $V(x, y)$  integrated with a
16     plain  $dy$  (the ``0.5/|1-ksi|``-over-[0, 2] structure of the CG PDFs needs the  $ksi$ 
17     integral of ``_build_pdf_matrix`` instead, so PDF kernels use that, not this).
18
19     The matrix rows live on ``lc_x_ls`` (the output light-cone  $x$  grid) and its
20     columns on ``quasi_y_ls`` (the input quasi  $x$  grid), so the caller forms the matched
21     distribution as ``lightcone = matrix @ quasi``. The two are independent open grids;
22     ``quasi_y_ls`` defaults to ``lc_x_ls``. The loop fills the off-diagonal
23     ( $x \neq y$ ) entries from ``density``; the LO  $\delta(x - y)$  is a linear-interpolation
24     stencil from the  $y$  grid onto each  $x$ , so it survives when the grids are staggered and
25     collapses to the identity when they coincide; the plus prescription makes every  $y$ 
26     column integrate to zero and thereby restores the  $x = y$  singularity;
27     ``diagonal_extra(y)`` adds an optional finite diagonal conversion term.
28     Returns ``identity - alpha_s C_x/(2 pi) * matrix * dy`` -- the matched distribution
29     is one matrix product, with no matrix inverse anywhere.
30     """
31     x_grid = np.asarray(lc_x_ls, dtype=float)
32     y_grid = np.asarray(x_grid if quasi_y_ls is None else quasi_y_ls, dtype=float)
33
34     if x_grid.ndim != 1:
35         raise ValueError("`lc_x_ls` must be a 1D array.")

```

```

36     if y_grid.ndim != 1 or y_grid.size < 2:
37         raise ValueError("`quasi_y_ls` must be a 1D array with at least 2 points.")
38     if np.any(np.abs(y_grid) <= eps):
39         # Every density here carries a 1/y (PDF: the dy/|y| measure; DA: V's own 1/y),
40         # so a y = 0 column is singular for either.

```

6.8 规划工具目录 (planning/agent.py)

```

1 PLAN_TOOL_CATALOG = {
2     "load_manifest": "Return the current in-memory manifest candidate and planned output paths.",
3     "check_manifest_draft": "Run deterministic manifest checks that tolerate incomplete drafts.",
4     "list_stage_parameter_gaps": "Return structured missing parameters and missing input roles for configured stages,
   ↪ including allowed options or example values.",
5     "inspect_correlator_h5_files": "Summarize HDF5, NPY, and NPZ datasets/arrays referenced by inputs.correlators.",
6     "plan_correlator_h5_conversions": "Detect source HDF5/NPY/NPZ files that need conversion to the standard
   ↪ correlator HDF5 layout.",
7     "apply_correlator_conversion_mapping": "Apply one user-confirmed correlator conversion. Args must be {
   ↪ correlator_id, datasets:[{source,target,axis_order?,index?,transpose?}, ...]}; never pass source/target at
   ↪ top level.",
8     "validate_candidate_manifest": "Run strict schema, DAG, and stage-local validation on the current candidate.",
9     "apply_manifest_patch_to_candidate": "Apply guarded JSON Patch edits to the in-memory candidate after validation."
   ↪ ,
10    "build_quick_full_candidates": "Build quick/full manifest candidates and validate their strict schema.",
11 }

```

7 参考源清单

参考源	类型	内容/作用
PLAN.md:5-12	仓库	项目定位与设计动机 (LaMET 五步流程、agent 分工)
PLAN.md:14-44	仓库	correlator/renorm 设计意图与策略 选择自由度的讨论
PLAN.md:142-156	仓库	extrapolation 外推项与外推策略
README.md:1-29	仓库	核心思想 (manifest DAG)、五 stage 顺序
README.md:73-136	仓库	NetCDF 中间产物格式与读写
README.md:185-335	仓库	manifest 参数语义 (fit_scope/ model_average/normalization 等)
README.md:373-427	仓库	自重整化公式与 workflow
README.md:575-659	仓库	Quick Start 命令、plan 模式、部分 运行

参考源	类型	内容/作用
README.md:669-829	仓库	标准 correlator HDF5 格式、CLI 后端用法
README.md:931-965	仓库	Agent Workflow 七步、Current Status
PROJECT_LOG.md:5-17	仓库	脚手架初始化与扁平布局
PROJECT_LOG.md:21-49	仓库	core+ 五 stage 布局、correlator 首实作
PROJECT_LOG.md:273-281	仓库	correlator 12→4 工具 agentic 收敛
PROJECT_LOG.md:343-367	仓库	job-DAG manifest 迁移、NetCDF 手递
PROJECT_LOG.md:445-451	仓库	全局重采样元数据
PROJECT_LOG.md:502-531	仓库	交互规划与 LLM 规划循环
PROJECT_LOG.md:526-568	仓库	自重整化方案演进
PROJECT_LOG.md:644-665	仓库	严格 stage 参数契约、pointwise ratio
PROJECT_LOG.md:871-889	仓库	review 文献背景注入与锚定排名
PROJECT_LOG.md:976-1026	仓库	prompts.md 资源化、包布局迁移
PROJECT_LOG.md:1049-1078	仓库	run 校验失败 plan 回退、matching 网格警告
__main__.py:111-135	仓库	validate 命令实现
__main__.py:240-350	仓库	run 命令实现与后端解析
agent.py:183-284	仓库	LLM/工具循环（必需序列、action 分派）
agent.py:561-660	仓库	run_agent 主循环、store 注入与输出登记
agent.py:857-989	仓库	各 stage 报告/叠加图生成
manifest.py:47-68	仓库	体积/动量解析、物理动量 GeV 推导
manifest.py:71-113	仓库	RunMetadata 模型与重采样约束
manifest.py:385-529	仓库	AnalysisManifest 模型与 DAG 校验
manifest.py:532-658	仓库	运动学派生 derive_job_kinematics
manifest_params.py:31-43	仓库	递归参数合并
manifest_params.py:46-80	仓库	STAGE_PARAM_CONTRACTS 注册表
core/data.py:23-51	仓库	EnsembleInfo 与动量格点换算
core/data.py:65-126	仓库	EnsembleData 构造与 xarray 构建

参考源	类型	内容/作用
core/data.py:148-204	仓库	NetCDF 序列化/反序列化
core/resampling.py:52-81	仓库	bin/jackknife/bootstrap 实现
core/resampling.py:129-165	仓库	共享 bootstrap indices、配置重采样
core/llm.py:16-55	仓库	ACTION_SCHEMA、 PROVIDERS 表
core/llm.py:504-546	仓库	make_llm_session 四后端路由
core/tools.py:199-288	仓库	工具解析/必需序列/输入校验
core/tools.py:357-372	仓库	prepare_tool_args 参数注入
core/prompting.py:12-27	仓库	系统提示与 action 输出格式
core/stages.py:1-16	仓库	stage id→ 包名映射
planning/core.py:121-174	仓库	JSONC 解析与宽松 manifest 加载
planning/core.py:642-867	仓库	确定性草稿检查与严格校验复用
planning/agent.py:48-73	仓库	PLAN_ACTION_SCHEMA 与工具目录
planning/agent.py:1155-1200	仓库	run_interactive_plan 入口
stages/correlator/functions.py:440-465	仓库	3pt/2pt 比值谱分解模型
stages/correlator/functions.py:5777-5781	仓库	correlator STAGE_TOOLS 注册表
stages/renorm/functions.py:792-806	仓库	自重整化 $g(z)$ 拟合函数
stages/renorm/functions.py:1649-1657	仓库	renorm STAGE_TOOLS 注册表
stages/fourier/functions.py:2709	仓库	run_fourier_transform 工具
stages/fourier/functions.py:3283-3290	仓库	fourier STAGE_TOOLS 注册表
stages/matching/functions.py:899-905	仓库	matching STAGE_TOOLS 注册表
stages/extrapolation/functions.py:148-227	仓库	run_extrapolation 设计与外推项构造
stages/review/functions.py:293-399	仓库	文献锚定排名
stages/review/functions.py:578-585	仓库	write_review 工具
kernels.py:58-102	仓库	beta/alphas_nloop 耦合运行
kernels.py:361-420	仓库	build_matching_matrix 列求和 plus 处方
kernels.py:681-760	仓库	公有 quark 核函数族
examples/sample_manifest.jsonc:1-45	仓库	注释清单设计说明与 metadata
examples/sample_manifest.jsonc:46-110	仓库	输入关联函数/kernels 声明
examples/fake_data/generate_fake_data.py:24-40	仓库	假数据规模/参数
examples/fake_data/utils.py:228-312	仓库	bin/bootstrap/jackknife 工具
papers/lamet_kb/pipeline.py:26-60	仓库	知识库抓管线
papers/lamet_kb/scoring.py:18-40	仓库	打分归一化与词项匹配

参考源	类型	内容/作用
<code>papers/docs/lamet_rubric.md:1-25</code>	仓库	core/secondary/irrelevant 标注规则
<code>papers/README.md:1-30</code>	仓库	知识库使用说明
<code>lamet_literature/README.md:1-30</code>	仓库	文献标注流程与标签语义
<code>pyproject.toml:10-36</code>	仓库	依赖组与 console script
<code>tests/unit/test_stage_core.py:20-33</code>	仓库	提示组装测试样例

参考背景补充 (类型 = 参考): `refer/AGENTS.md` (PyQCD 主仓库参考说明, 提供本库在 `refer` 下的背景)。未引入: 无其它参考目录。

8 结论与局限

结论

三视角结论: **结构**——`lamet-agent` 是“入口薄、核心深、知识库独立”的三层仓库, 入口 (`__main__`/`agent`) 只做编排, 数值全在 `stages/*/functions.py` 与 `core/`。**关系**——五 stage 通过 job-DAG 的 role-named inputs 串成数据链, `planning` 与 `stages` 共用 `manifest` 模式, `review` 单方向消费知识库文献背景。**思路**——把 LaMET 分析中“需要物理判断”的两步委托给 LLM 决策、“相对固定”的三步软件化, 再用 `manifest` 校验 + 必需工具序列保证“agent 自由决策不越界”。**主题结论:** agent 测量全流程 A1-A15 重现完毕, 数值基元与模式层经 23/117/121 项测试实测稳定; 因本 checkout 缺可运行 `manifest`, 48 项依赖 `example` 的集成测试未通过 (非逻辑缺陷)。

参考背景

旁证: 主仓库 `refer/AGENTS.md` 描述 `refer` 目录按贡献者组织参考代码, `lamet-agent` 作为 `git-rep/` 外来参考仓库被归档只读——其价值定位是“上游参考实现”, `PyQCD` 主仓库 `pyqcd/renorm` 的移植源是 `zengch`, 而非本库 (与本库无代码依赖)。

局限与风险

局限与风险:

- 真实数值 `run` 未执行, 物理产物正确性 **未验证** (缺数据);
- `examples/*.json` 可运行 `manifest` 未入库, **未验证** `run` 全流程;
- `review` 阶段 LLM 文本输出与确定性检查的一致性仅靠提示约束;
- 本库非独立 `git` 仓库, `git` 历史不能独立反映 `lamet-agent` 演进, 以 `PROJECT_LOG.md` 为准。